Context & Scenario

You have **exactly 30 minutes** to build a prototype for a new shipping logistics software. The boss wants a system that can handle different types of packages (Standard, Overnight, Heavy) and calculate the total shipping bill for a customer's order.

Crucial Requirement: Your boss is paranoid about memory management and object integrity. You are **forbidden** from using `std::string` for the sender/receiver names. You must use `char*` (C-style strings) to manage the names manually to ensure you fully understand the Rule of Three (Destructor, Copy Constructor, Assignment Operator).

Step-by-Step Instructions (The App)

Open your IDE (VS Code, CLion, or an online C++ compiler). You must create the following architecture:

1. Create the Abstract Base Class: `Package`

- **Data Members:** `char* senderName`, `char* receiverName`, `double weight`.
- **Static Members:** `static int totalPackagesCreated`. This should increment by 1 every time a `Package` object is created.
- **Constructors:**
  - Parameterized constructor (takes sender name, receiver name, weight).
  - Copy constructor (Must perform a **Deep Copy** of the `char*` arrays).
- **Destructor:**
  - Must free the memory allocated for `senderName` and `receiverName`.
  - Must be `virtual` (Think: Why?).
- **Member Functions:**
  - `virtual double calculateCost() = 0;` (Pure virtual - making this class abstract).
  - `void display() const;` (Prints sender, receiver, weight).

2. Create Derived Classes (Public Inheritance)

- `TwoDayPackage`:
  - *Additional Data:* `double costPerKg`.
  - *Cost Logic:* `weight * costPerKg`.

- `OvernightPackage`:
  - *Additional Data:* `double costPerKg`, `double additionalFeePerKg`.
  - *Cost Logic:* `weight * (costPerKg + additionalFeePerKg)`.
- *Override the* `calculateCost()` *and* `display()` *functions accordingly.*

3. Create the Driver Logic (in `main.cpp`)

1. Create a dynamic array (or `std::vector`) of `Package*` pointers.
2. Add at least 2 `TwoDayPackage` objects and 2 `OvernightPackage` objects to this container.
3. Iterate through the container and use polymorphism to calculate the *total cost* of all packages in the batch.
4. Display the details of each package and the final total.

Deliverable 2: The Findings Report (Minimum 2 Pages)

After you finish the 30-minute sprint (or complete the code), you must write a detailed report analyzing your experience. **Do not just paste your code.** You must write an actual report discussing the engineering decisions.

Your report must be **at least 2 pages** long (standard font, double spaced). Use the following sections:

Page 1: Architectural Decisions & Implementation

- **Abstract Base Class Strategy:** Why did you choose `Package` as an abstract class? Explain the role of the `= 0` in `virtual double calculateCost() = 0;`.
- **The Memory Management Challenge:** Discuss the requirement to use `char*` instead of `std::string`. Explain the difference between **Shallow Copy** and **Deep Copy**. How did you implement the Copy Constructor to ensure that copying a `TwoDayPackage` didn't cause a double-free error later?
- **The Rule of Three:** Did you implement the Destructor, Copy Constructor, and Copy Assignment Operator? Explain what happens if you define a destructor but forget to implement the copy constructor in a class managing dynamic memory.

Page 2: Challenges, Polymorphism, and Reflection

- **Encountered Compilation Issues:** Did you face "pure virtual function call" errors? Ambiguity issues? Did you remember to make the Base destructor `virtual`? Explain the consequences if the Base destructor was not virtual.
- **Polymorphic Power:** Describe how you used the Base class pointer (`Package*`) to store different derived objects (`TwoDayPackage` and `OvernightPackage`) in the same container. How did the `vtable` (Virtual Table) help resolve the correct `calculateCost()` at runtime?
- **Static Data Usage:** How did you use `static int totalPackagesCreated`? How could this static variable help in a real-world inventory scenario?
- **Conclusion:** What is the single most important OOP concept (Encapsulation, Inheritance, or Polymorphism) you feel you mastered by doing this 30-minute sprint? How does this apply to real-world software engineering?

Important Instructions

- **Do not** look for solutions online. The purpose of this sprint is to make *mistakes* (like forgetting `delete[]` for the `char*`, or trying to dynamically cast the base pointer).
- Your **Findings Report** is the most valuable part of this assignment. If your code crashes due to a shallow copy bug, writing about *why* it crashed and how the Rule of Three solves it is considered excellent work.
- Upload your `.cpp`, `.h`, and `.txt` or `.docx` findings report to the Google Classroom link provided.

********END OF THE DOCUMENT********